

32)Bit stuffing Mechanism using C

Program:

```
#include <stdio.h>
```

```
void bitStuffing(int data[], int n) {  
    int stuffed_data[100];  
    int i, j = 0, count = 0;  
  
    for (i = 0; i < n; i++) {  
        stuffed_data[j] = data[i];  
        j++;  
  
        if (data[i] == 1) {  
            count++;  
        } else {  
            count = 0;  
        }  
  
        // If five consecutive 1s are encountered, stuff a 0  
        if (count == 5) {  
            stuffed_data[j] = 0;  
            j++;  
            count = 0; // Reset counter after stuffing  
        }  
    }  
  
    printf("Data after Bit Stuffing: ");  
    for (i = 0; i < j; i++) {  
        printf("%d", stuffed_data[i]);  
    }  
}
```

```
}  
printf("\n");  
}
```

```
void bitDestuffing(int stuffed_data[], int stuffed_len) {  
    int destuffed_data[100];  
    int i, j = 0, count = 0;  
  
    for (i = 0; i < stuffed_len; i++) {  
        destuffed_data[j] = stuffed_data[i];  
        j++;  
  
        if (stuffed_data[i] == 1) {  
            count++;  
        } else {  
            count = 0;  
        }  
  
        // If five consecutive 1s are encountered, skip the next stuffed 0  
        if (count == 5) {  
            i++; // Skip the next bit (which is the stuffed 0)  
            count = 0; // Reset counter  
        }  
    }  
  
    printf("Data after Bit De-stuffing: ");  
    for (i = 0; i < j; i++) {  
        printf("%d", destuffed_data[i]);  
    }  
}
```

```

    printf("\n");
}

int main() {
    int n, stuffed_len;
    int data[100], stuffed_data[100];
    int i;

    // --- Bit Stuffing Section ---
    printf("Enter number of bits: ");
    scanf("%d", &n);

    printf("Enter the bits (0/1):\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &data[i]);
    }

    bitStuffing(data, n);

    // --- Bit De-stuffing Section ---
    printf("\nEnter stuffed data length: ");
    scanf("%d", &stuffed_len);

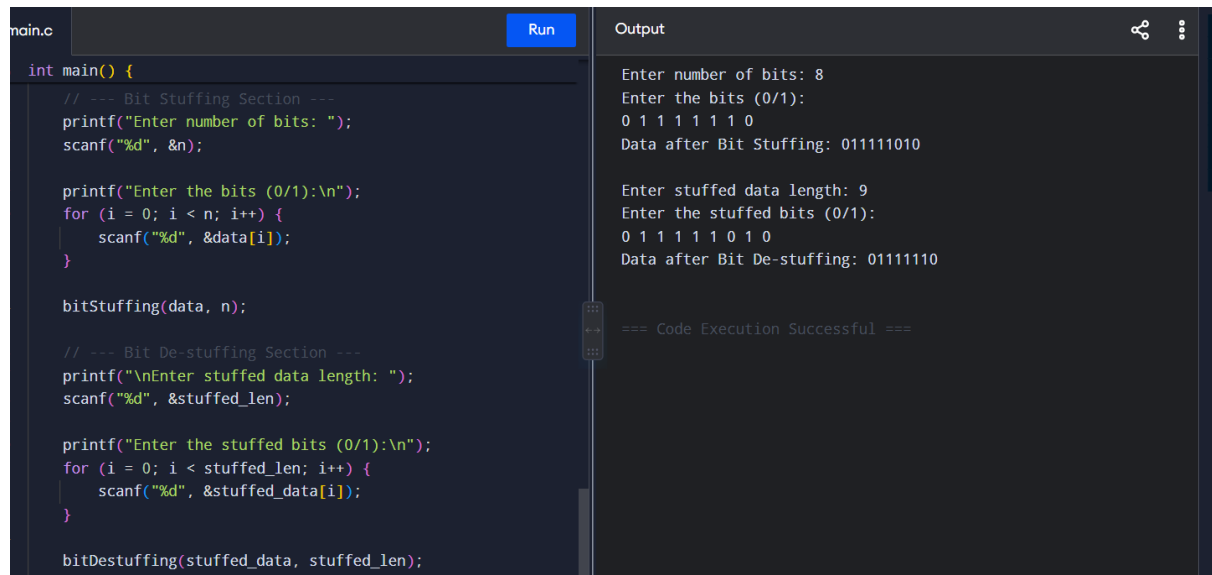
    printf("Enter the stuffed bits (0/1):\n");
    for (i = 0; i < stuffed_len; i++) {
        scanf("%d", &stuffed_data[i]);
    }

    bitDestuffing(stuffed_data, stuffed_len);
}

```

```
    return 0;
}
```

Output:



The screenshot shows a C++ IDE with a file named 'main.c'. The code implements a bit stuffing algorithm. It prompts the user to enter the number of bits (8) and the bits themselves (0 1 1 1 1 1 0). It then performs bit stuffing, resulting in the stuffed data '011111010'. Next, it prompts for the stuffed data length (9) and the stuffed bits (0 1 1 1 1 1 0 1 0). Finally, it performs bit de-stuffing, resulting in the original data '01111110'. The output window shows the same prompts and results, and a message indicating successful code execution.

```
main.c Run Output
int main() {
    // --- Bit Stuffing Section ---
    printf("Enter number of bits: ");
    scanf("%d", &n);

    printf("Enter the bits (0/1):\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &data[i]);
    }

    bitStuffing(data, n);

    // --- Bit De-stuffing Section ---
    printf("\nEnter stuffed data length: ");
    scanf("%d", &stuffed_len);

    printf("Enter the stuffed bits (0/1):\n");
    for (i = 0; i < stuffed_len; i++) {
        scanf("%d", &stuffed_data[i]);
    }

    bitDestuffing(stuffed_data, stuffed_len);
}
```

Enter number of bits: 8
Enter the bits (0/1):
0 1 1 1 1 1 0
Data after Bit Stuffing: 011111010

Enter stuffed data length: 9
Enter the stuffed bits (0/1):
0 1 1 1 1 1 0 1 0
Data after Bit De-stuffing: 01111110

=== Code Execution Successful ===